

CSEN604: Data Bases II

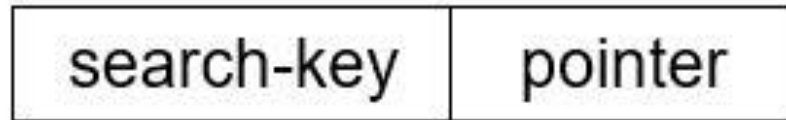
Mohamed Karam Gabr

Conventional Indices

Spring 2025

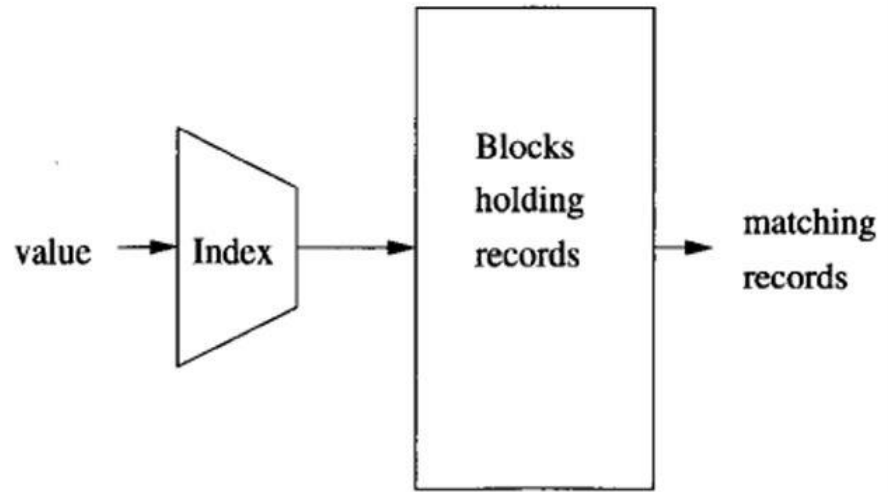
Basic Concepts

- Indexing mechanisms used to speed up access to desired data.
- **Search Key** - attribute to set of attributes used to look up records in a file.
- An **index file** consists of records (called **index entries**) of the form



- Index files are typically much smaller than the original file
- Two basic kinds of indices:
 - **Ordered indices:** search keys are stored in sorted order
 - **Hash indices:** search keys are distributed uniformly across “buckets” using a “hash function”.

Why Use Index



- A single-level index is an auxiliary file that makes it more efficient to search for a record in the data file
- An index has to be sorted according to a certain indexing field.
- Index structures provide secondary access paths.
- Any field can be used as an index.
- Multiple indices can be constructed.
- Index files are small and can be retrieved with less block accesses.

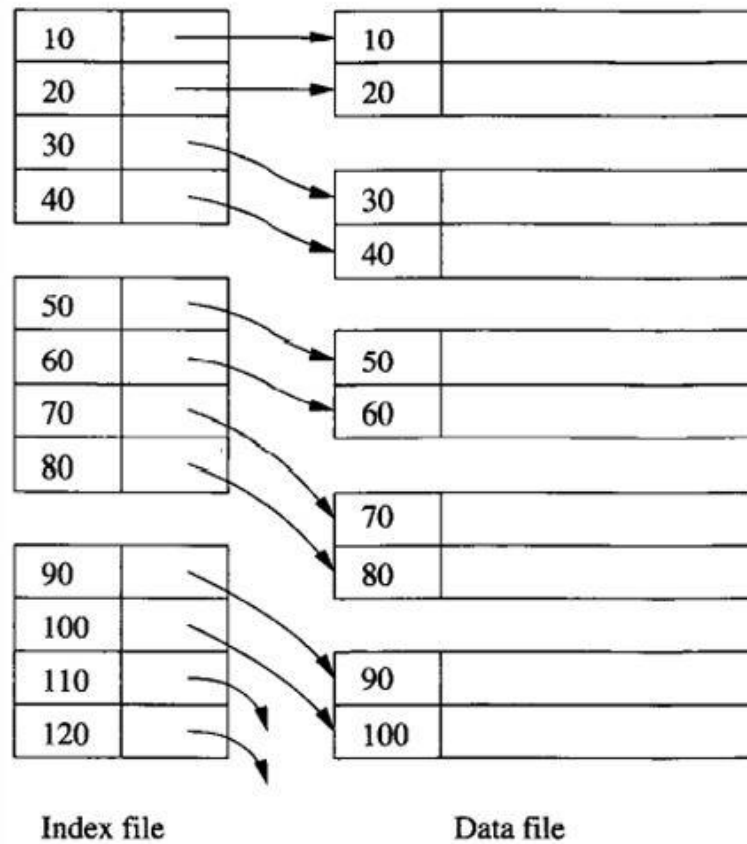
Key vs. Index vs. Clustering

- **Key:** logical concept.
- **Index:** a secondary storage data structure.
- **Clustering:** disk/page organizational issue According to which column the data is stored in pages.
- The clustering (i.e. sorting) column does not have to be the key column. Any column could be used in clustering the data in the pages of the table.

Types of Indices

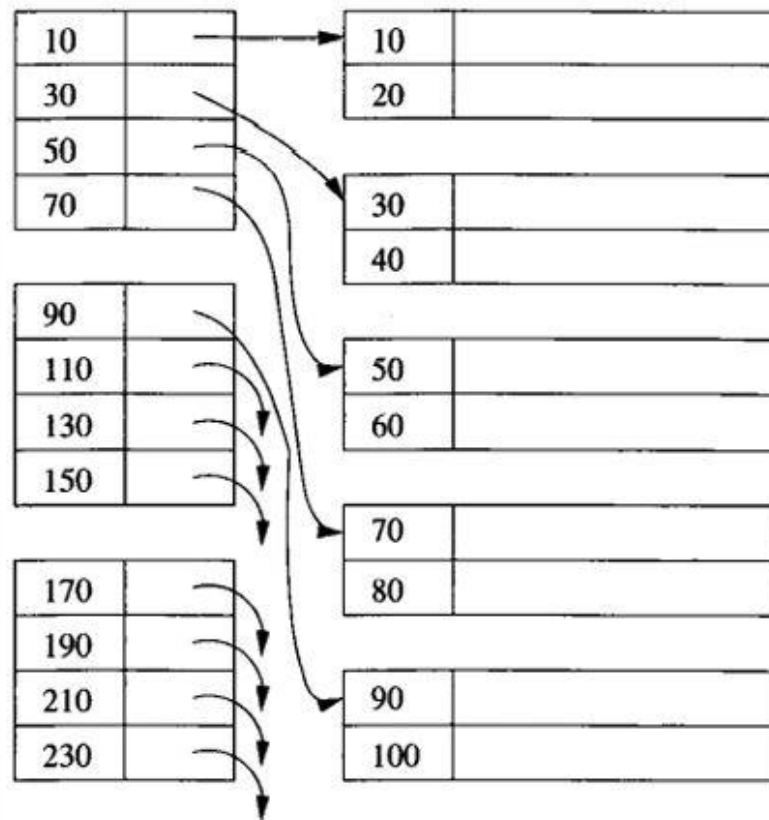
- Indices can be characterized as dense or sparse.
 - **Dense index** has an index entry for every search key value (and hence every record) in the data file.
 - **Sparse** (or nondense) **index** has index entries for only some of the search values.
- Single-level Ordered Indices:
 - Primary Indices (can be either dense or sparse).
 - Secondary Indices (Must only be dense).
- Multi-level Indices.

Dense Index



- A sequence of blocks holding only the keys of the records and pointers to the records.
- Dense index can be used for any search key even if not sorted.
- Index blocks of the dense index maintain these keys in the same sorted order as in the file.

Sparse Index

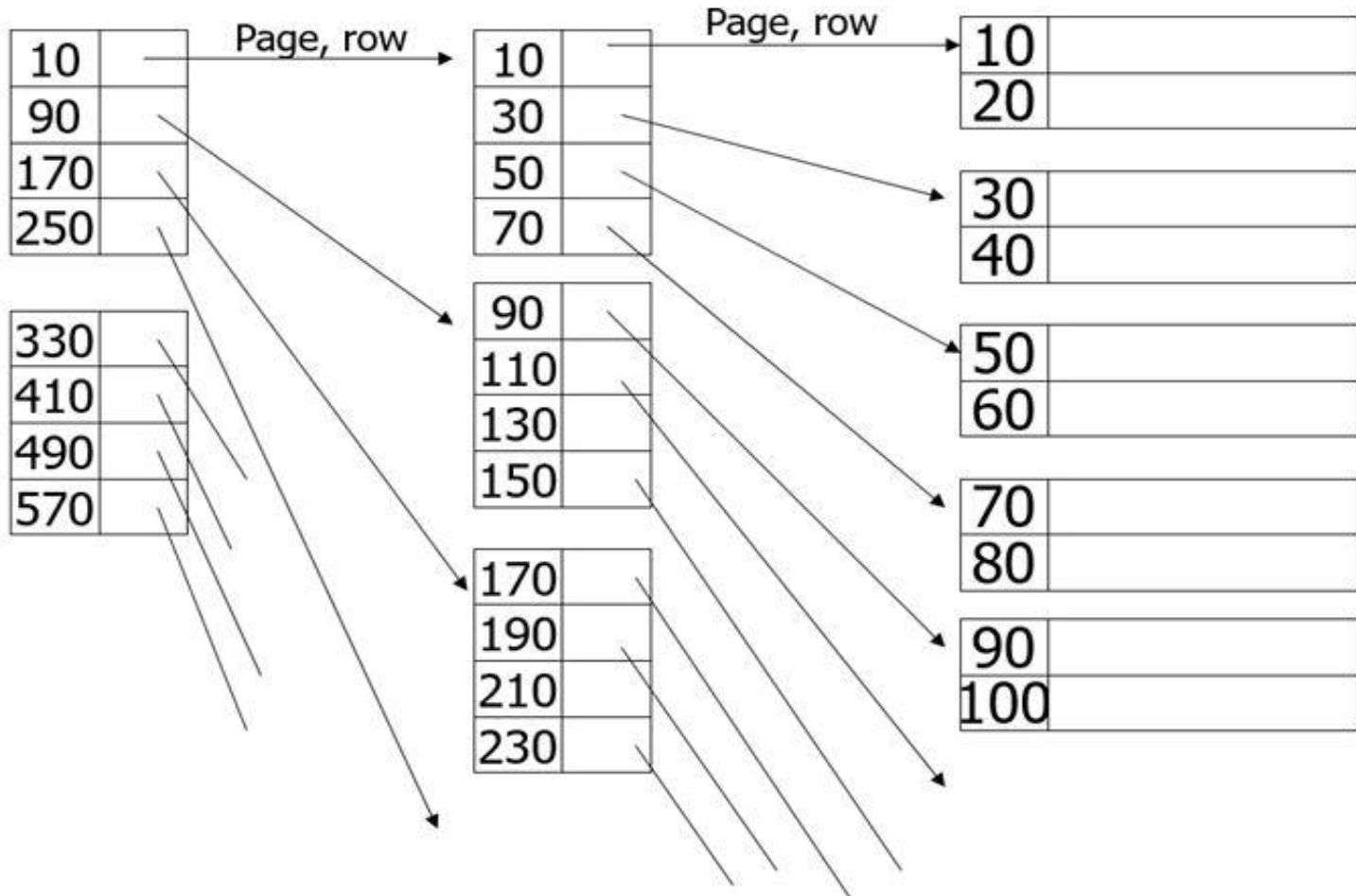


- A sparse index has only one key-pointer pair per block of the data file.
- Can only use a sparse index if the data file is sorted by the search key

Multi-level sparse Index

Sparse 2nd level

Relation



Dense vs. Sparse

- **Dense:**
 - Can find out if a record doesn't exist from index without reading data block.
 - Takes more space.
 - Can tell if any record exists without accessing relation file.
- **Sparse:**
 - Less maintenance overhead for insertion, and deletions.
 - Slower compared to the dense Index for locating records.
 - Less index space per record.
 - Can keep more of index in memory.

Primary Index

- The index is stored in memory, and the relation is on the HD.
- The relation is sorted on the field of the index.
- Can have only one primary index on a relation.
- Note: index is smaller in size.
- Additional link tells where is the next record on the HD.
- For Primary Dense Index, every record must be in the Index.
- For Primary Sparse Index, not every key have to appear in the index, but must access the relation file even if the record is not present.

Primary Dense Index

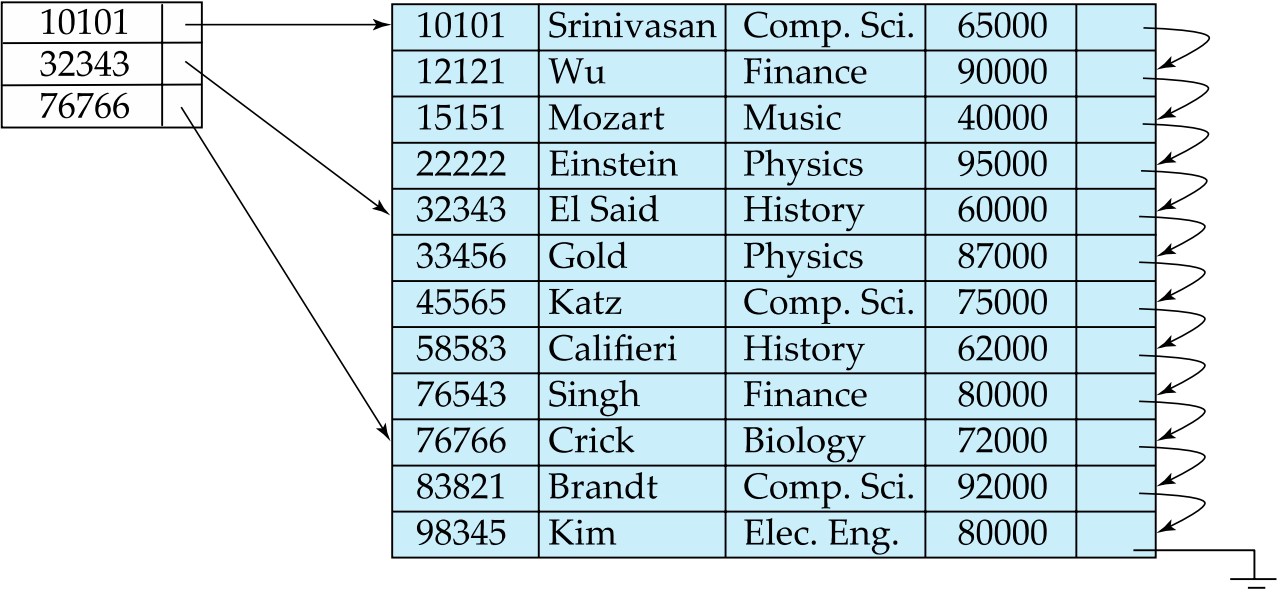
- Without Duplicated

10101		10101	Srinivasan	Comp. Sci.	65000	
12121		12121	Wu	Finance	90000	
15151		15151	Mozart	Music	40000	
22222		22222	Einstein	Physics	95000	
32343		32343	El Said	History	60000	
33456		33456	Gold	Physics	87000	
45565		45565	Katz	Comp. Sci.	75000	
58583		58583	Califieri	History	62000	
76543		76543	Singh	Finance	80000	
76766		76766	Crick	Biology	72000	
83821		83821	Brandt	Comp. Sci.	92000	
98345		98345	Kim	Elec. Eng.	80000	

- With Duplicates

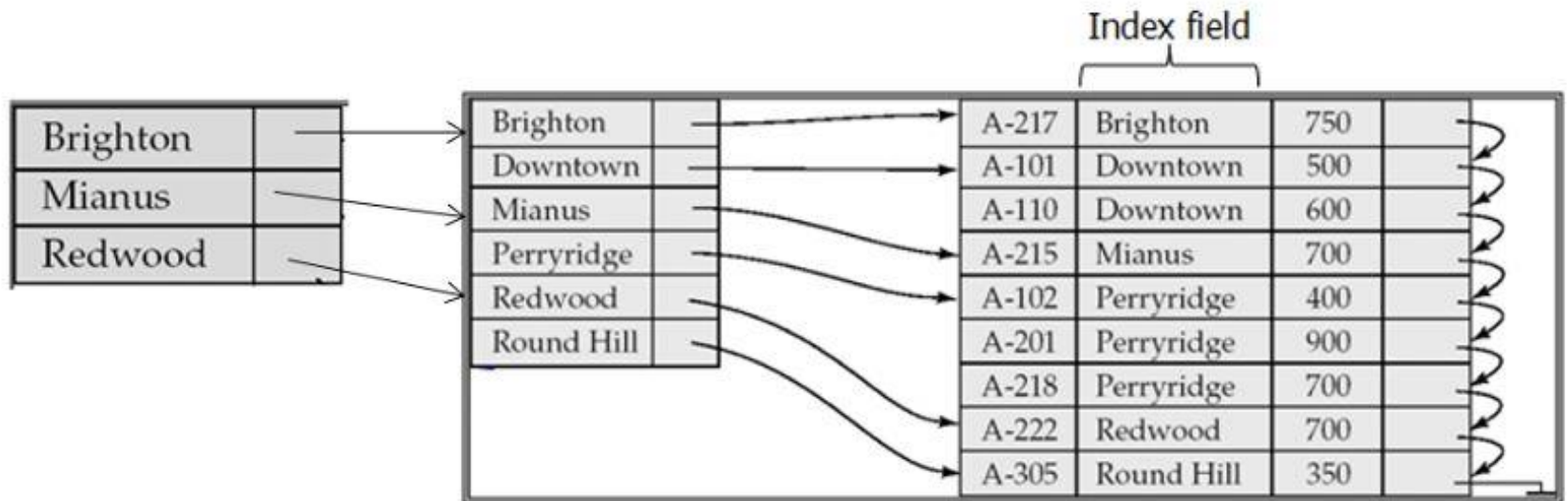
Biology		76766	Crick	Biology	72000	
Comp. Sci.		10101	Srinivasan	Comp. Sci.	65000	
Elec. Eng.		45565	Katz	Comp. Sci.	75000	
Finance		83821	Brandt	Comp. Sci.	92000	
History		98345	Kim	Elec. Eng.	80000	
Music		12121	Wu	Finance	90000	
Physics		76543	Singh	Finance	80000	
		32343	El Said	History	60000	
		58583	Califieri	History	62000	
		15151	Mozart	Music	40000	
		22222	Einstein	Physics	95000	
		33465	Gold	Physics	87000	

Primary Sparse Index



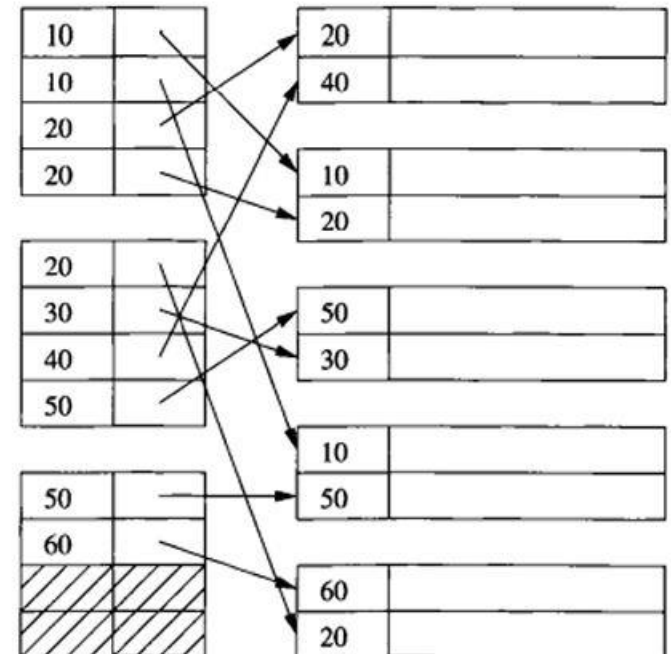
Multi Level Primary Index

- When you do not have enough memory to load all of the primary index.
- Points to primary dense or sparse index.



Secondary Index

- Secondary index is always created on non-primary field.
- It is a dense index.
- It may be on a field which is a candidate key and has a unique value in every record, **OR** a non-key with duplicate values.
- There can be many secondary indices.
- How to handle duplicates in the column?



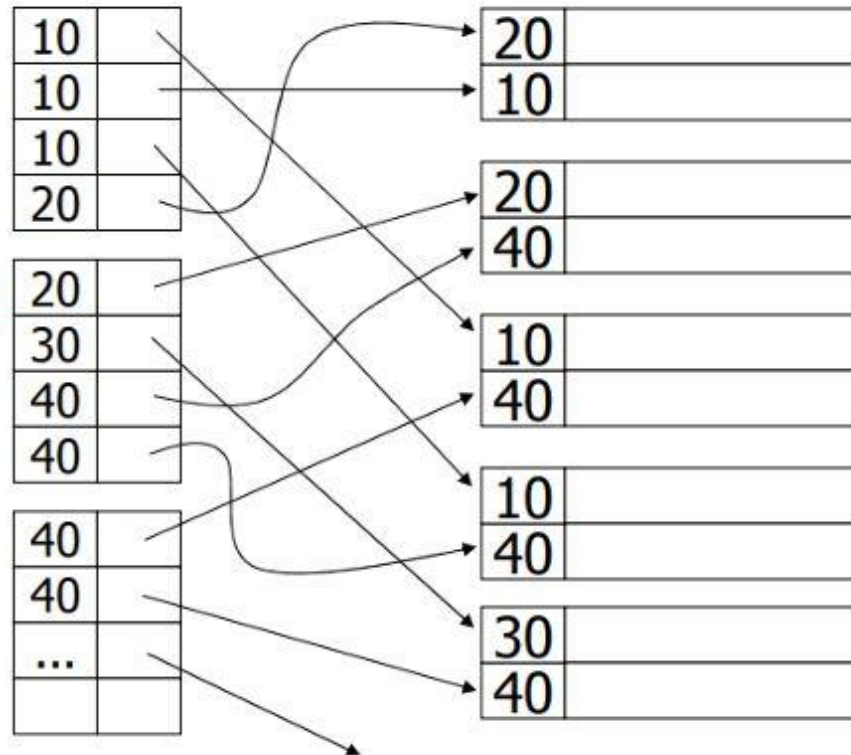
Option One:

Several index entries with the same $K(i)$ values

Problem:

excess overhead!

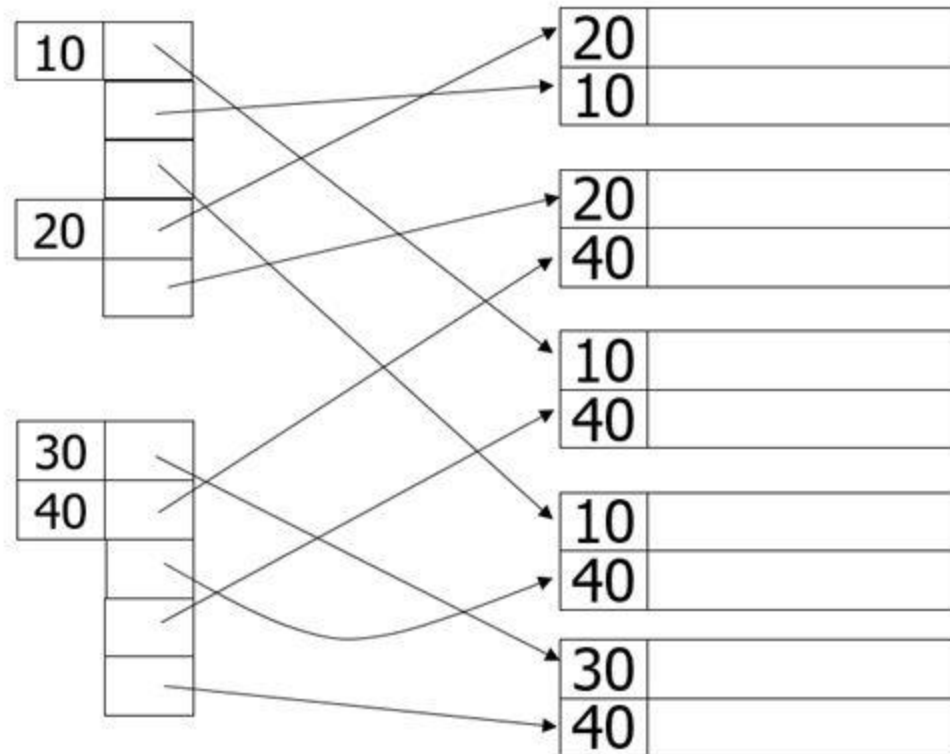
- disk space
- search time



Option Two:

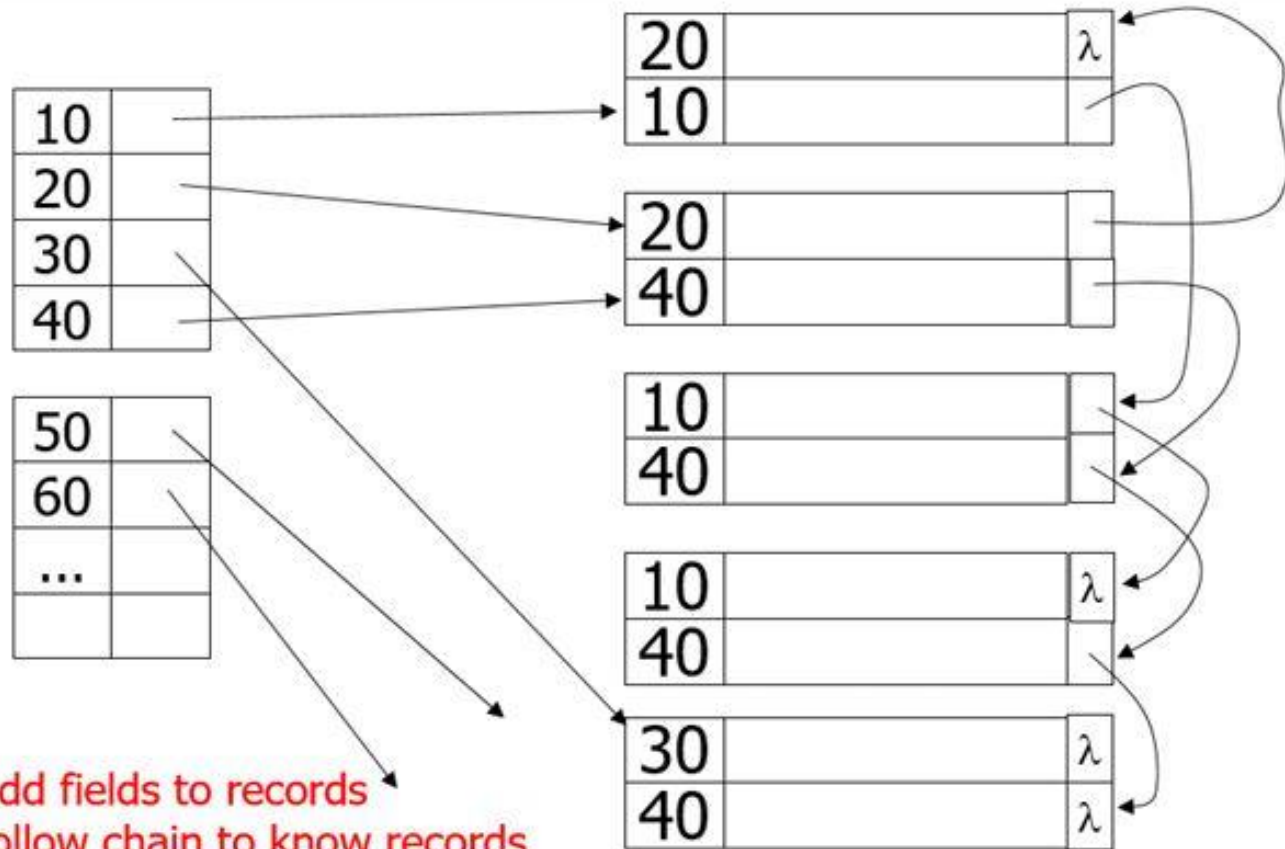
Use Linked list for each key

Problem:
variable size
records in
index!



Option Three:

Chain records with same key



Problems:

- Need to add fields to records
- Need to follow chain to know records
- Maintenance nightmare!

Clustering Index

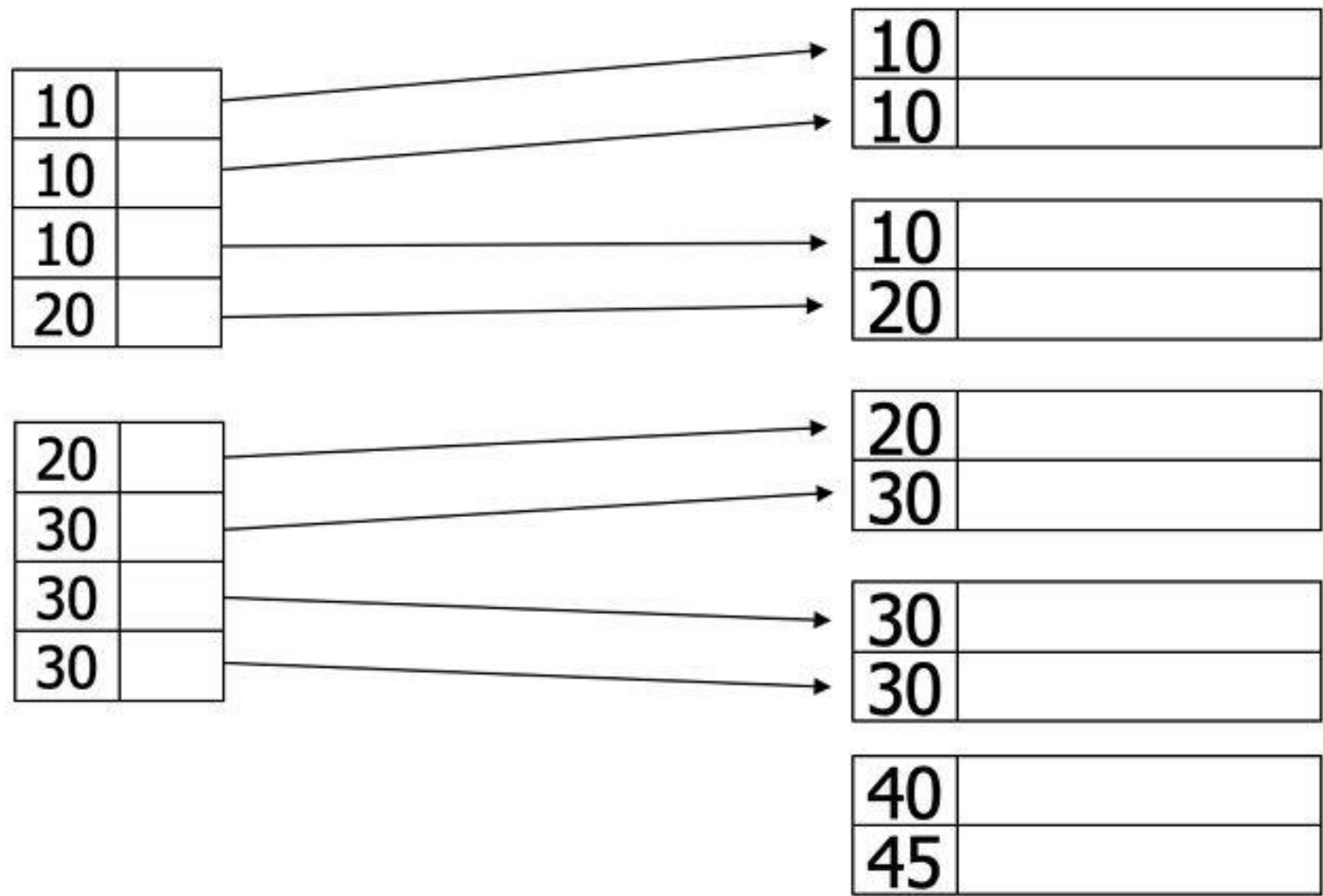
- Clustering defines the page content in terms of order tuples.
- Index and relation match in ordering.
- Only one clustered index can be created on a given database table.
- Clustered indices can greatly increase overall speed of retrieval, but usually only where the data is accessed sequentially in the same or reverse order of the clustered index, or when a range of items is selected.
- A clustered index is just another name for what we called before a primary index.

Non-Clustering Index

- Logical ordering is specified by the index only.
- The physical order of the rows is not the same as the index order.
- The indexed columns are typically non-primary key columns (used in JOIN, WHERE, and ORDER BY clauses).
- There can be more than one non-clustered index on a database table.
- A non-clustered index is just another name for what we called before secondary index.

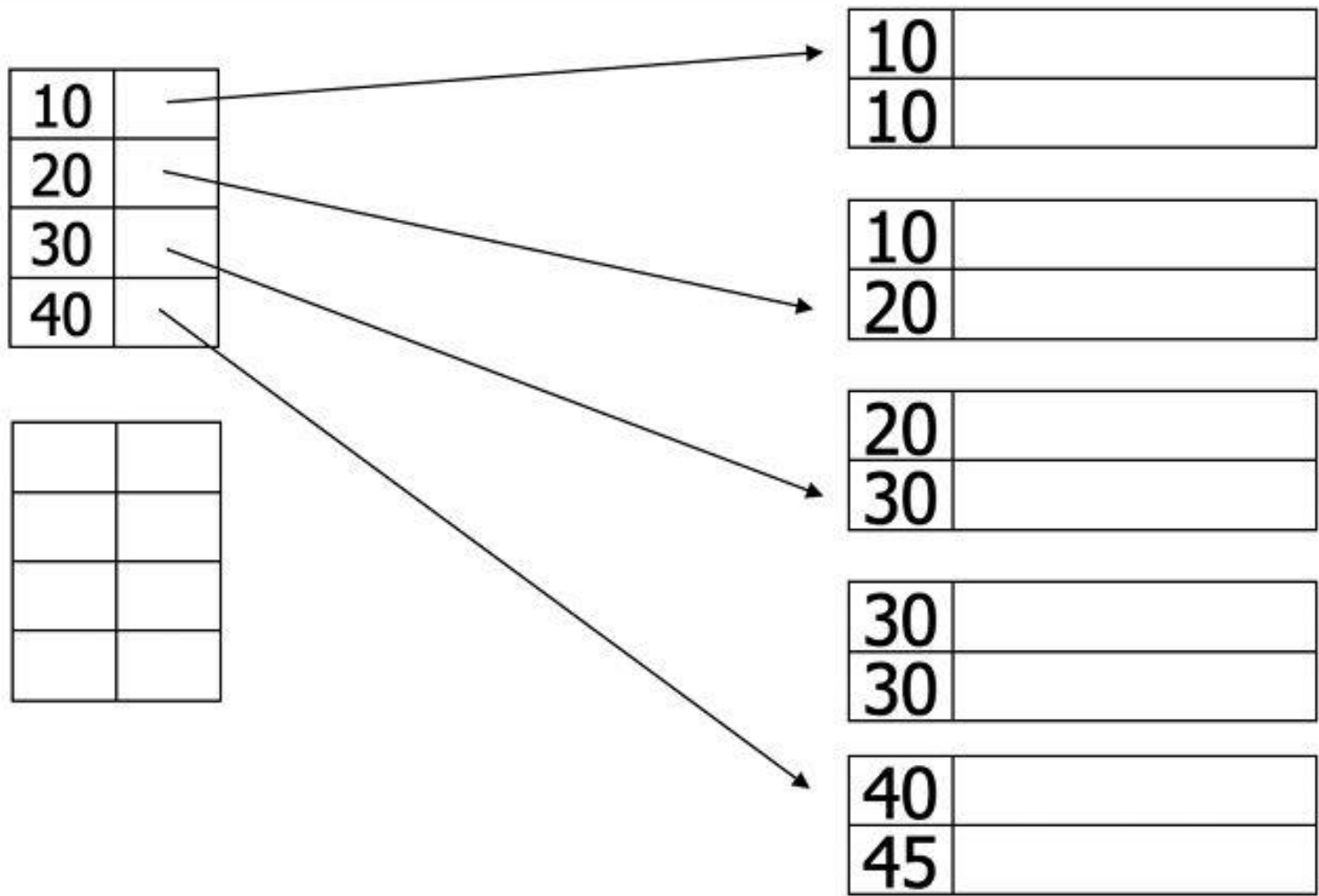
How to Handle Duplicates

In Dense Index (option one): Point to all values



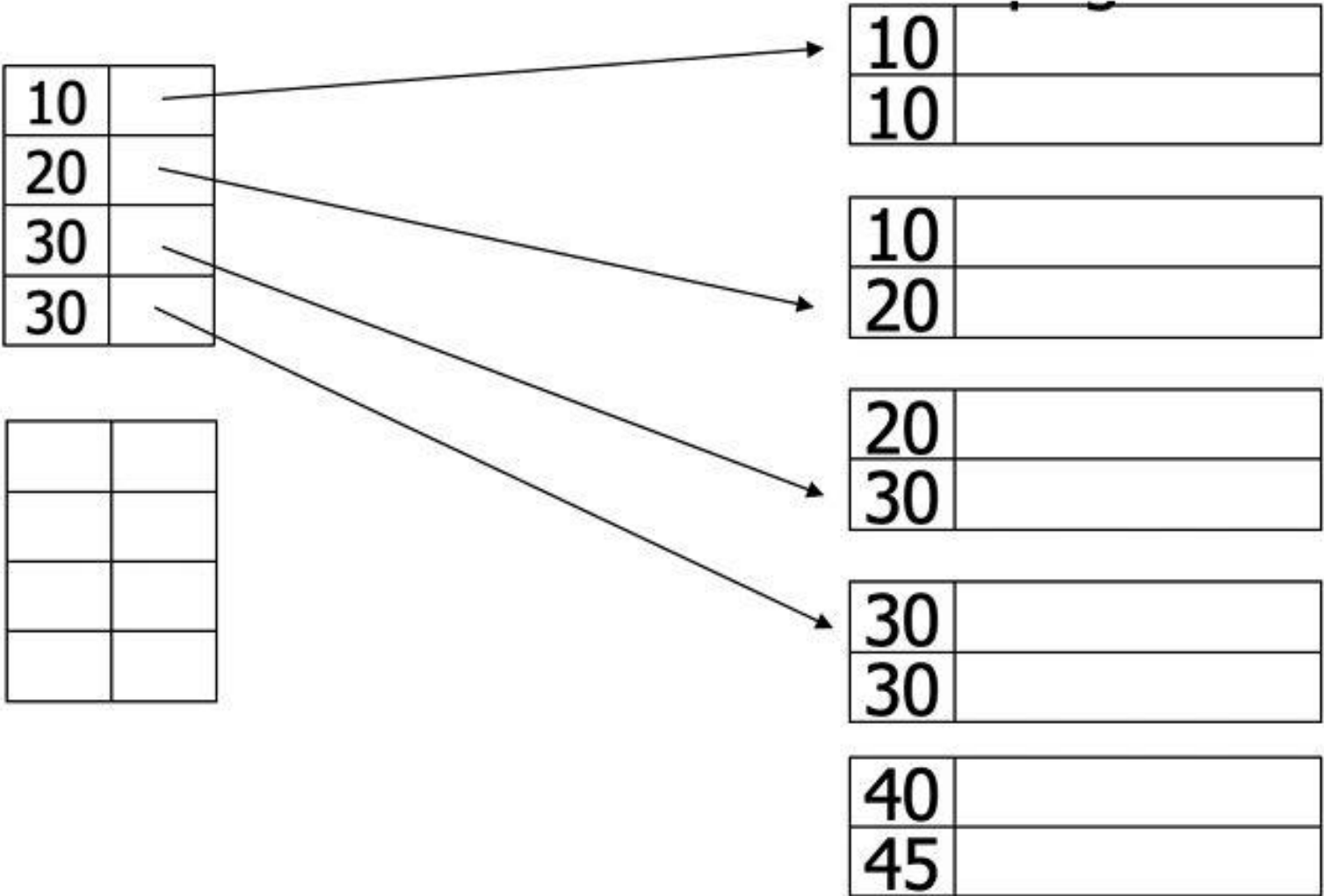
How to Handle Duplicates

In Dense Index (option two): Point to the first new value



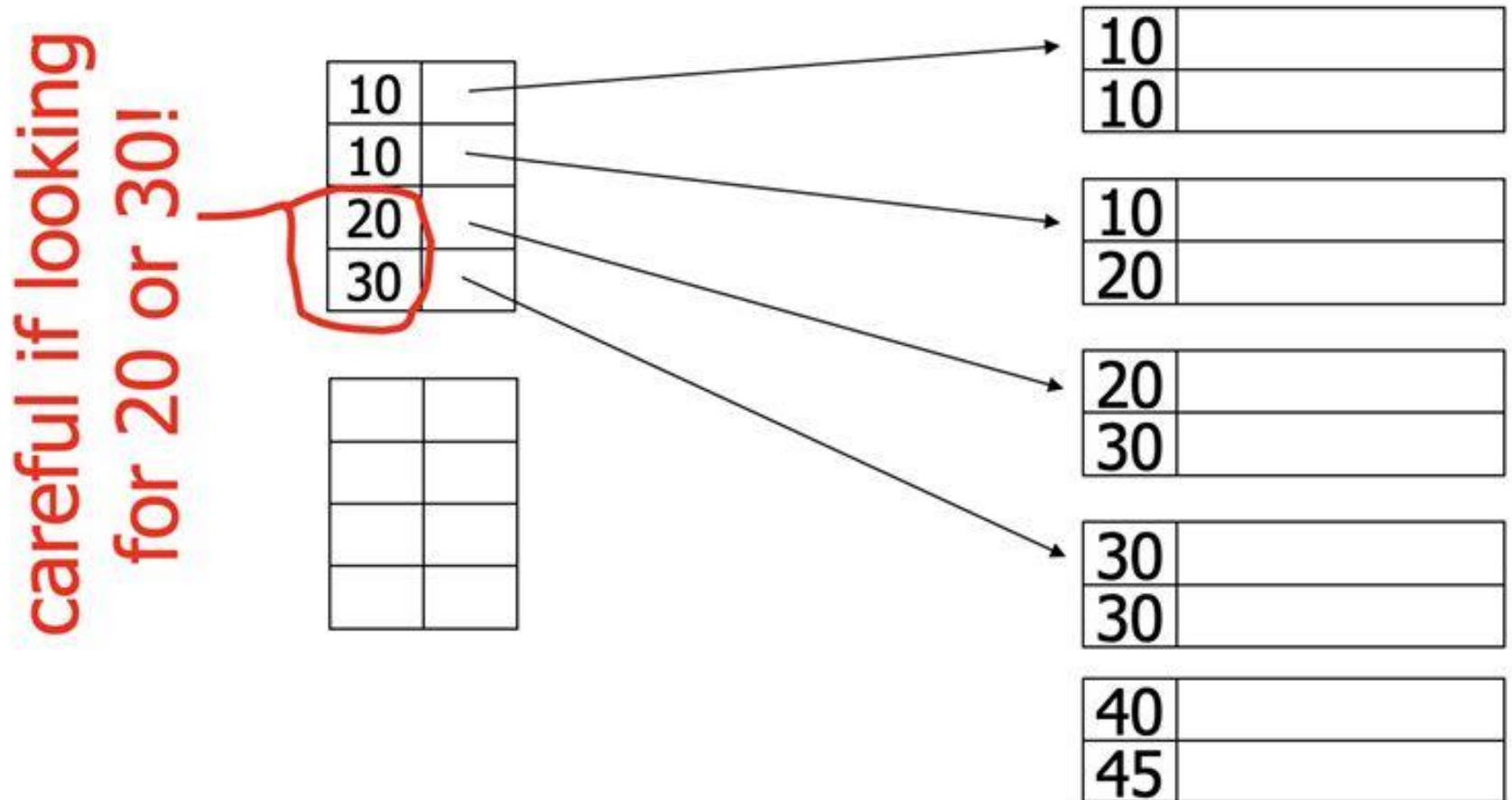
How to Handle Duplicates

In Dense Index (option three): At minimum; pick a value from each page



How to Handle Duplicates

In Sparse Index: Also pick a value from each page



Deletion from Sparse Index

Delete 40

10	
30	
50	
70	

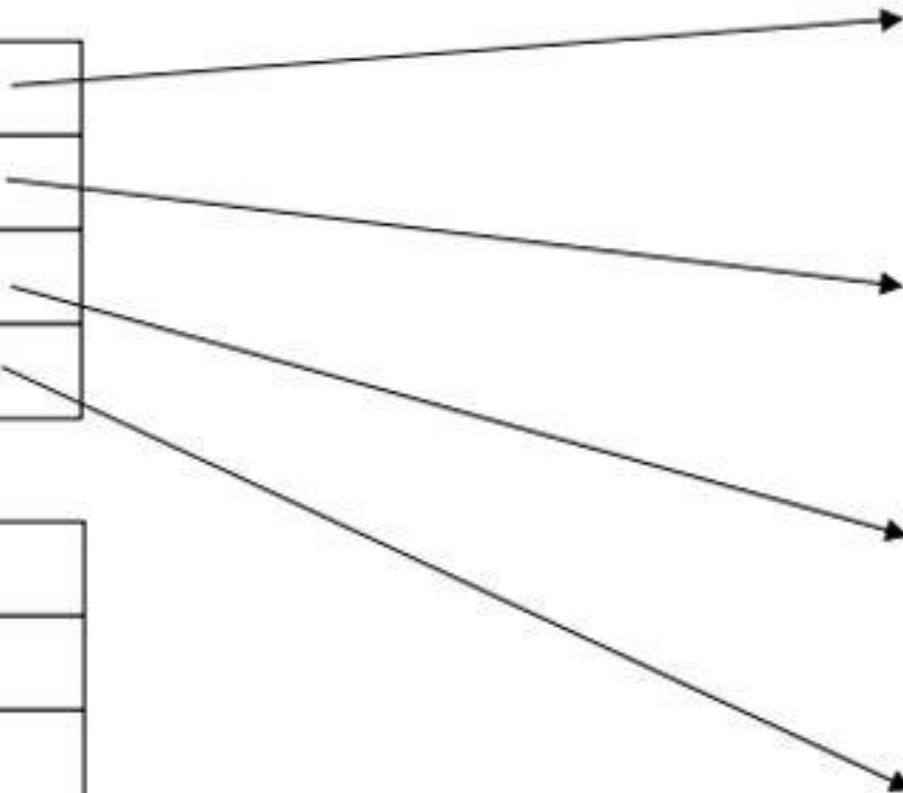
90	
110	
130	
150	

10	
20	

30	
40	

50	
60	

70	
80	



Deletion from Sparse Index

Delete 40

10	
30	
50	
70	

90	
110	
130	
150	

10	
20	

30	

50	
60	

70	
80	

Deletion from Sparse Index

Delete 30

10	
30	
50	
70	

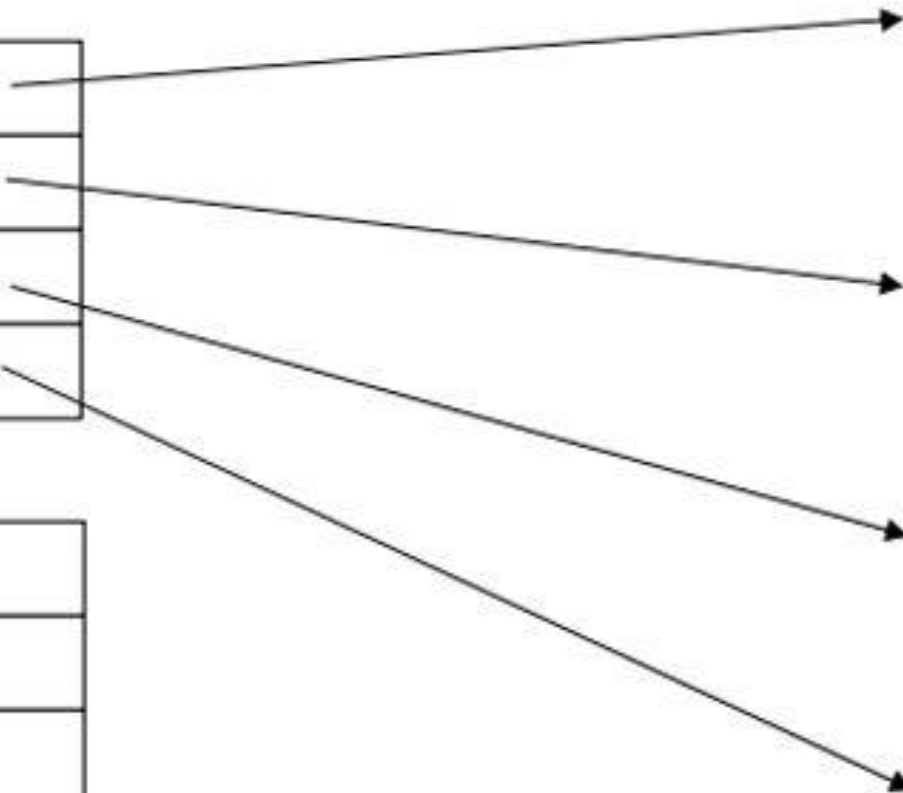
90	
110	
130	
150	

10	
20	

30	
40	

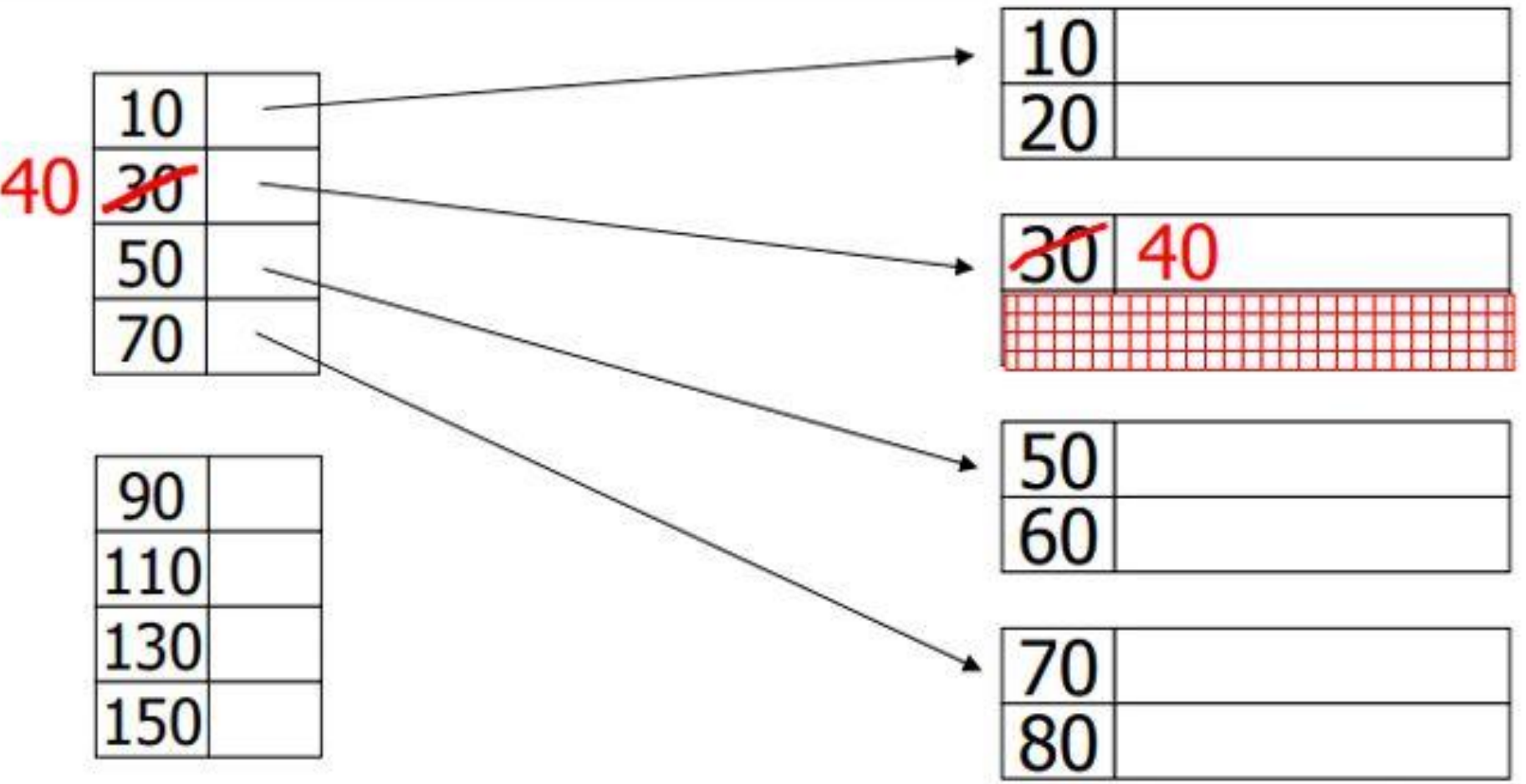
50	
60	

70	
80	



Deletion from Sparse Index

Delete 30



Deletion from Sparse Index

Delete 30 and 40

10	
30	
50	
70	

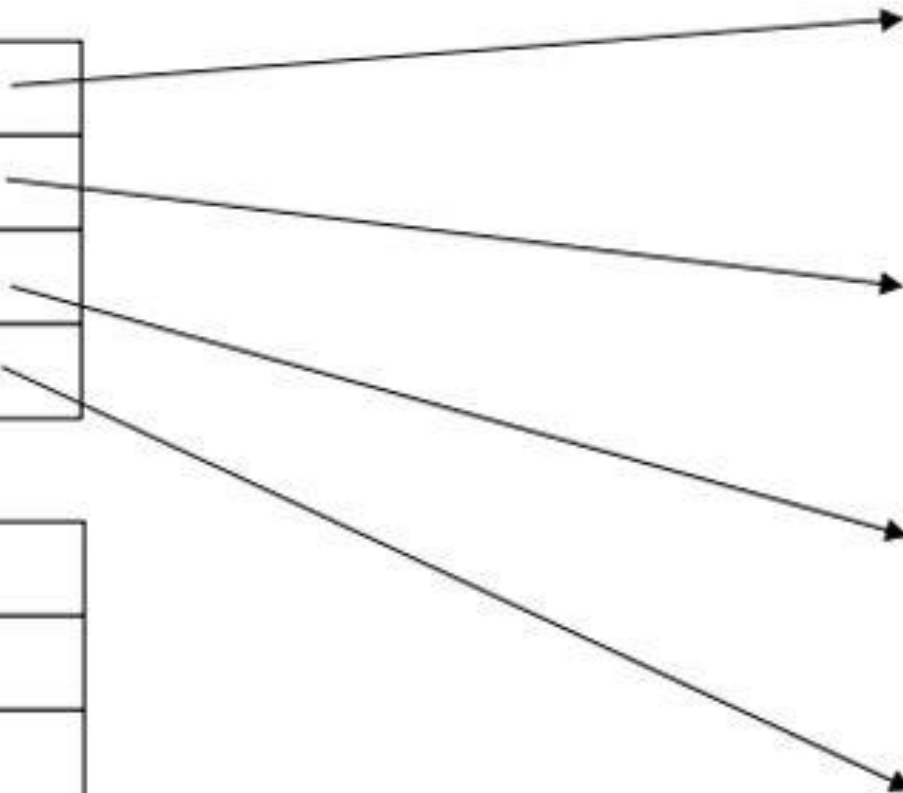
90	
110	
130	
150	

10	
20	

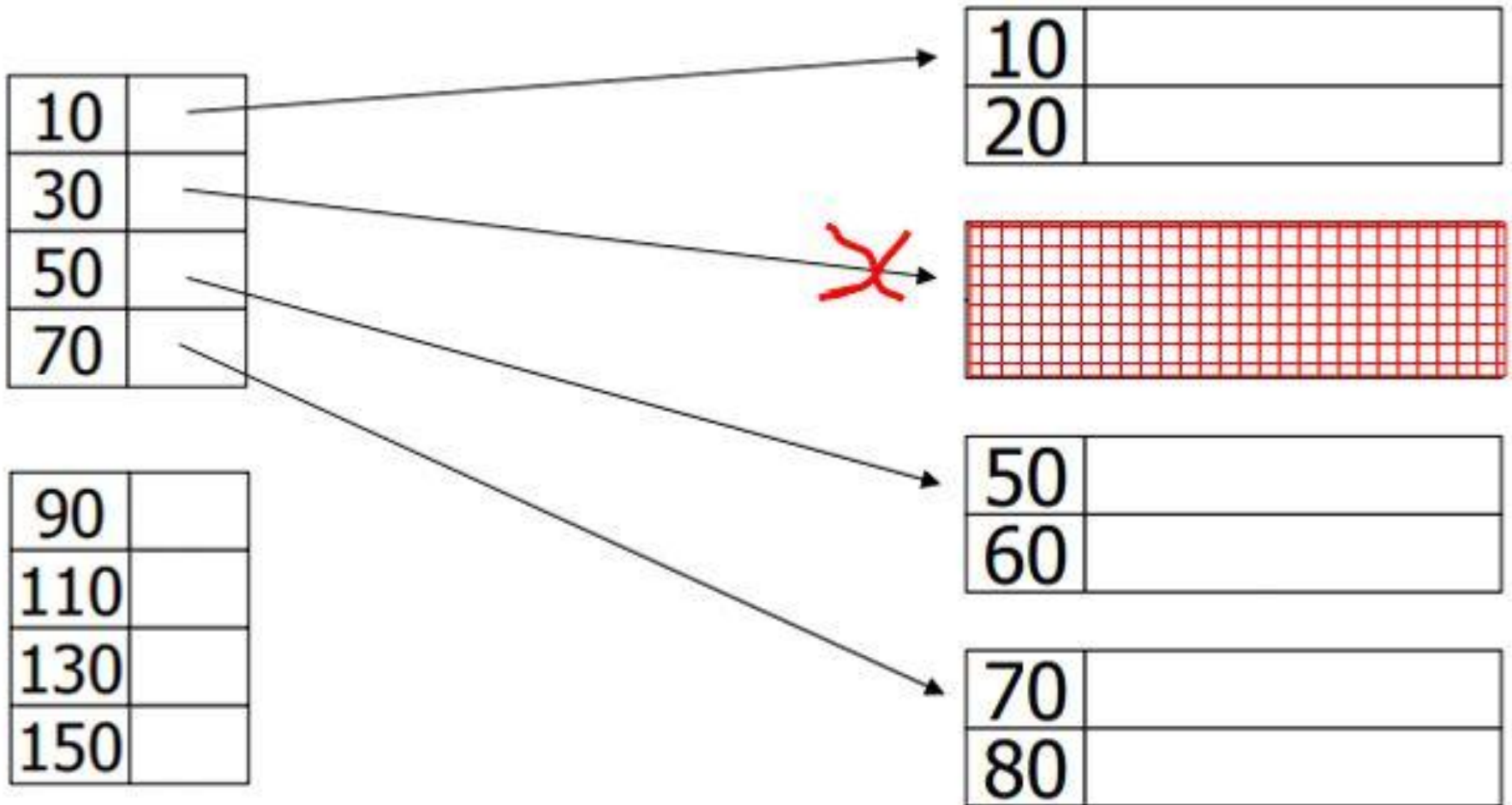
30	
40	

50	
60	

70	
80	

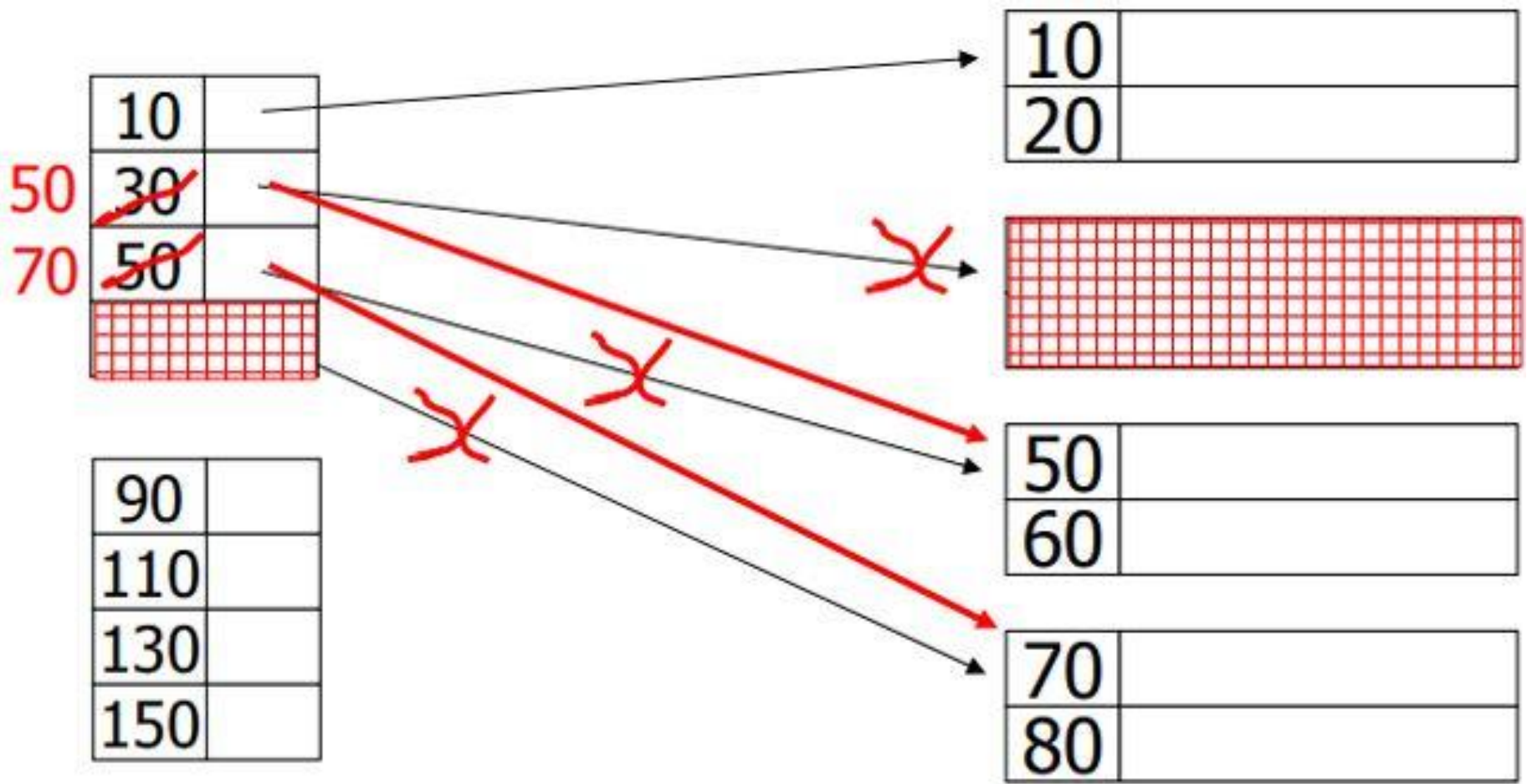


Delete 30 and 40



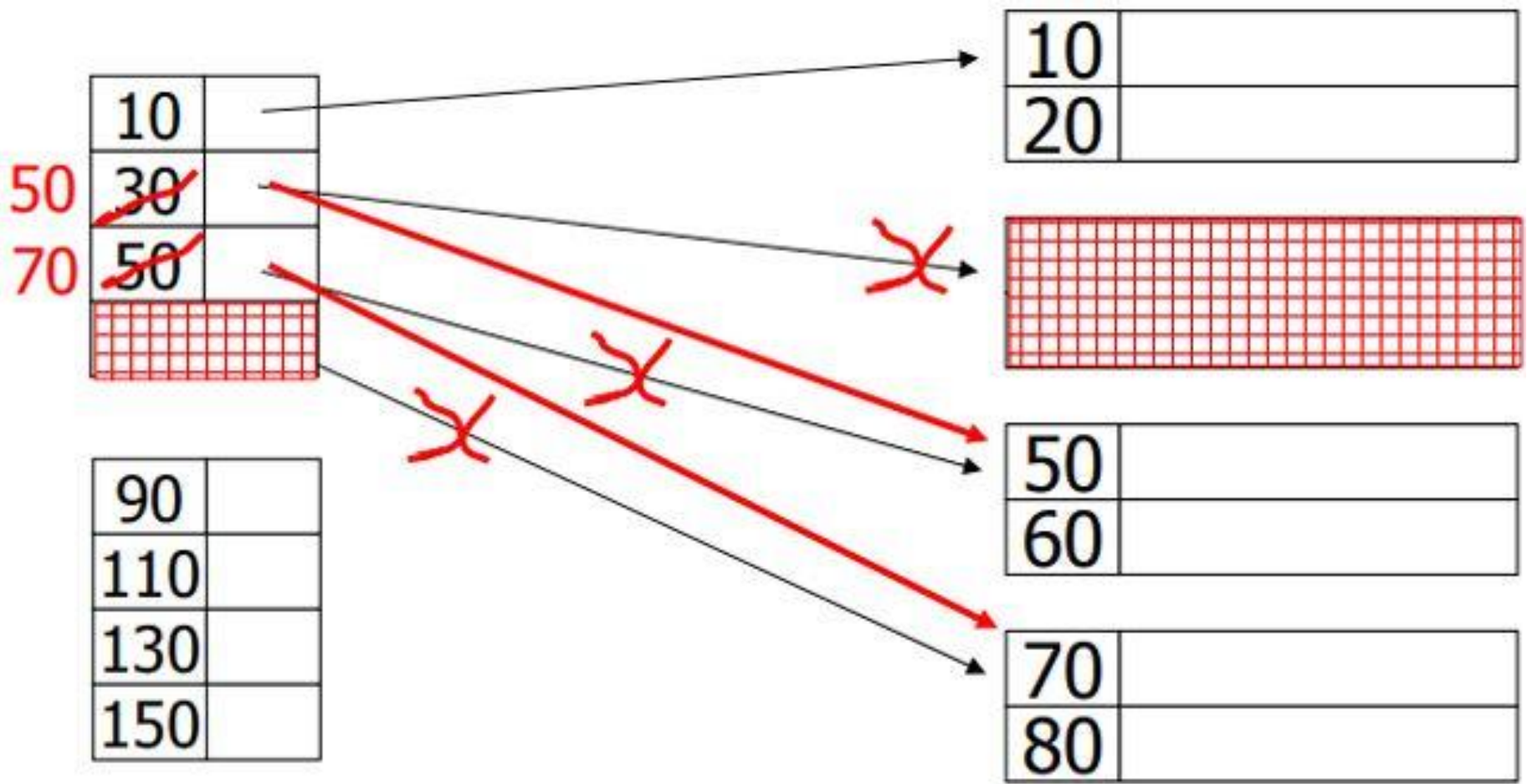
Deletion from Sparse Index

Delete 30 and 40



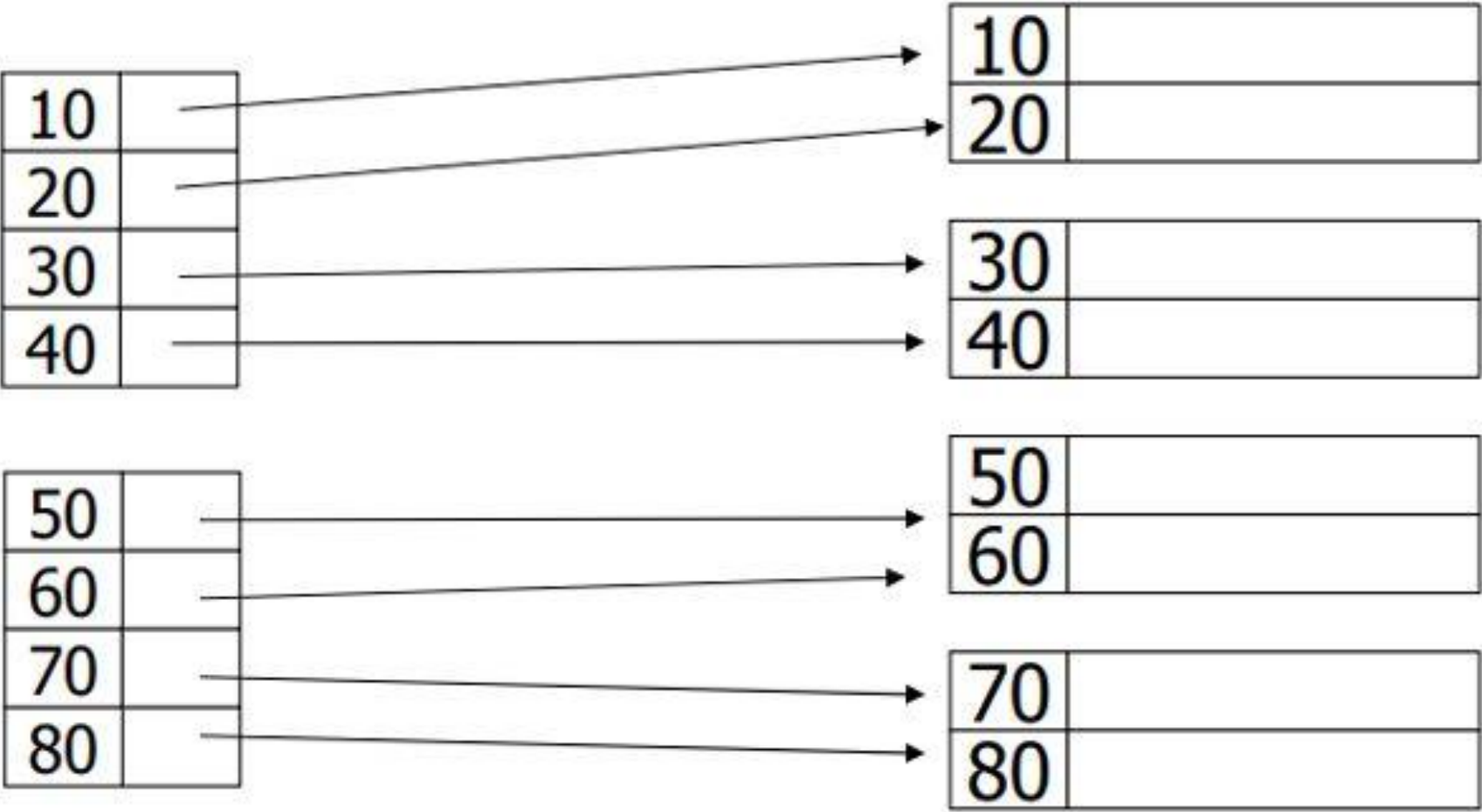
Deletion from Sparse Index

Delete 30 and 40



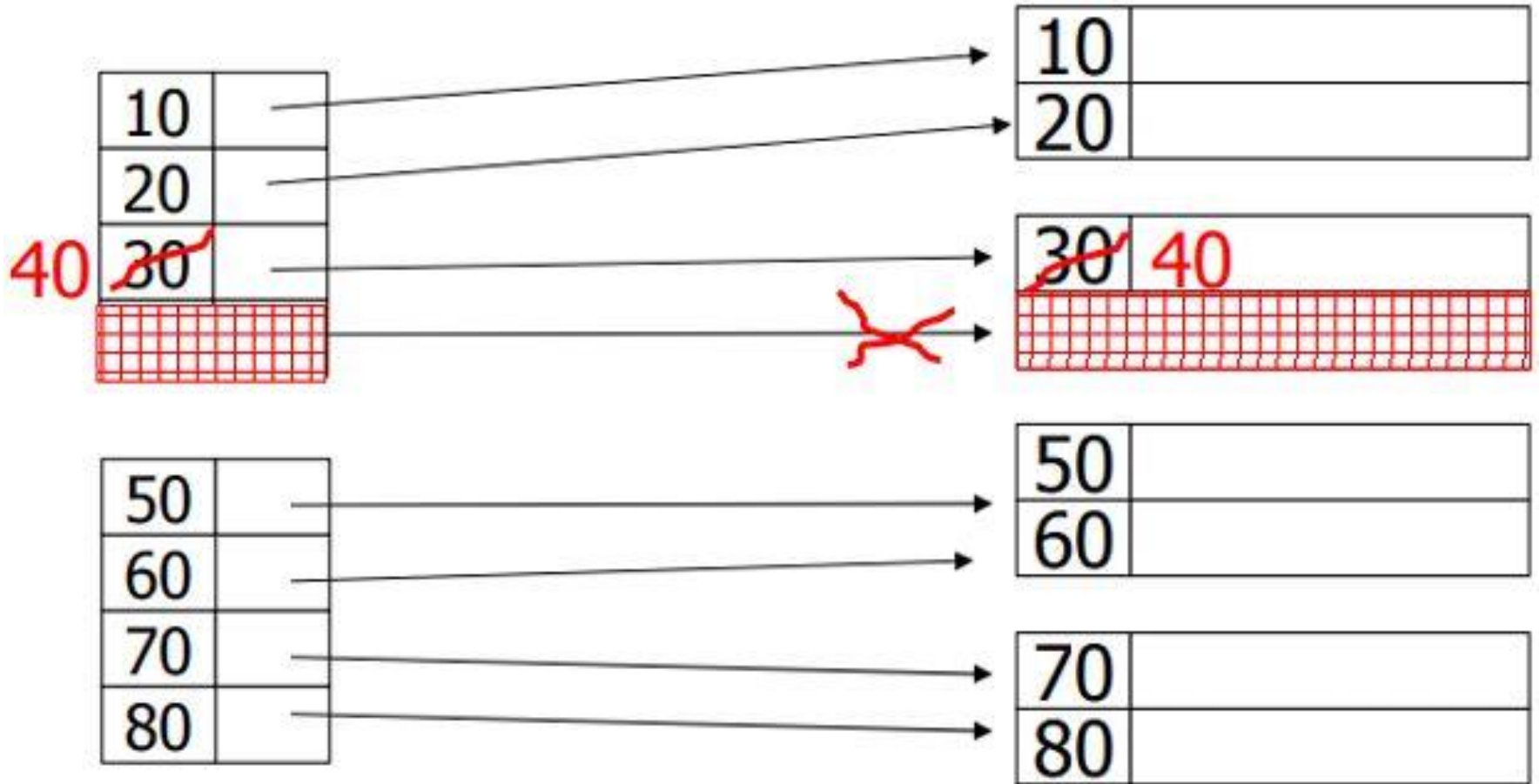
Deletion from Dense Index

Delete 30



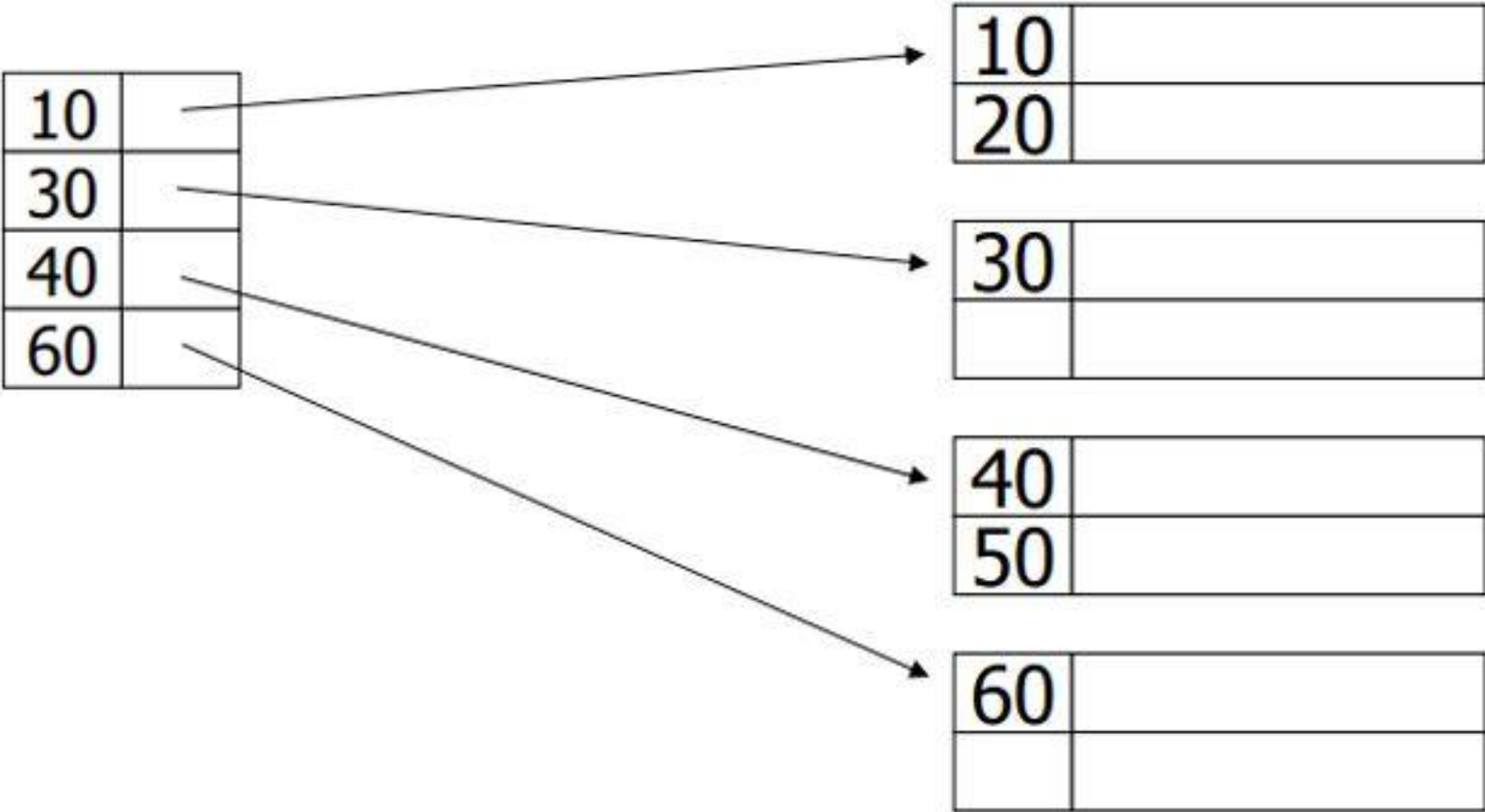
Deletion from Dense Index

Delete 30



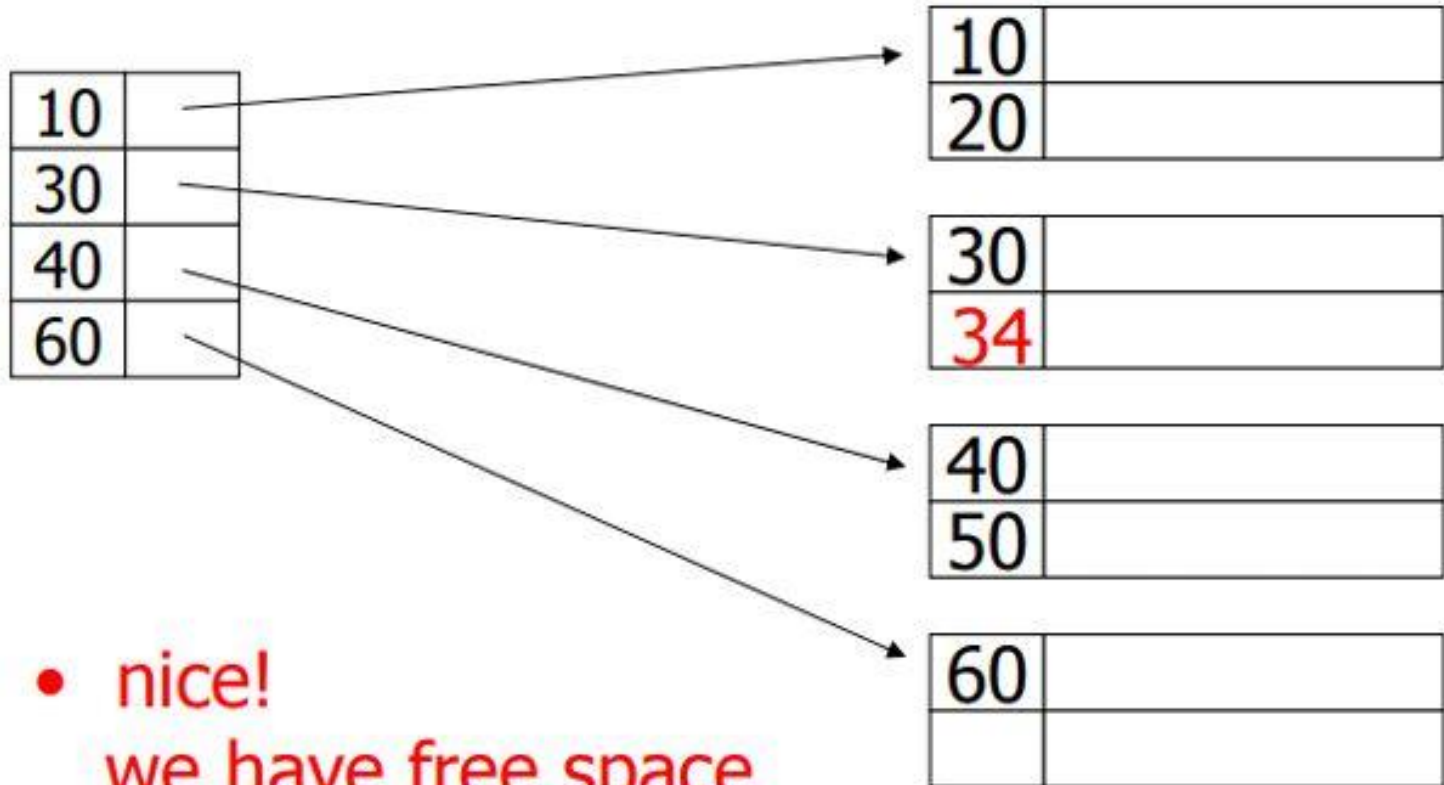
Insertion Into Sparse Index

Insert 34



Insertion Into Sparse Index

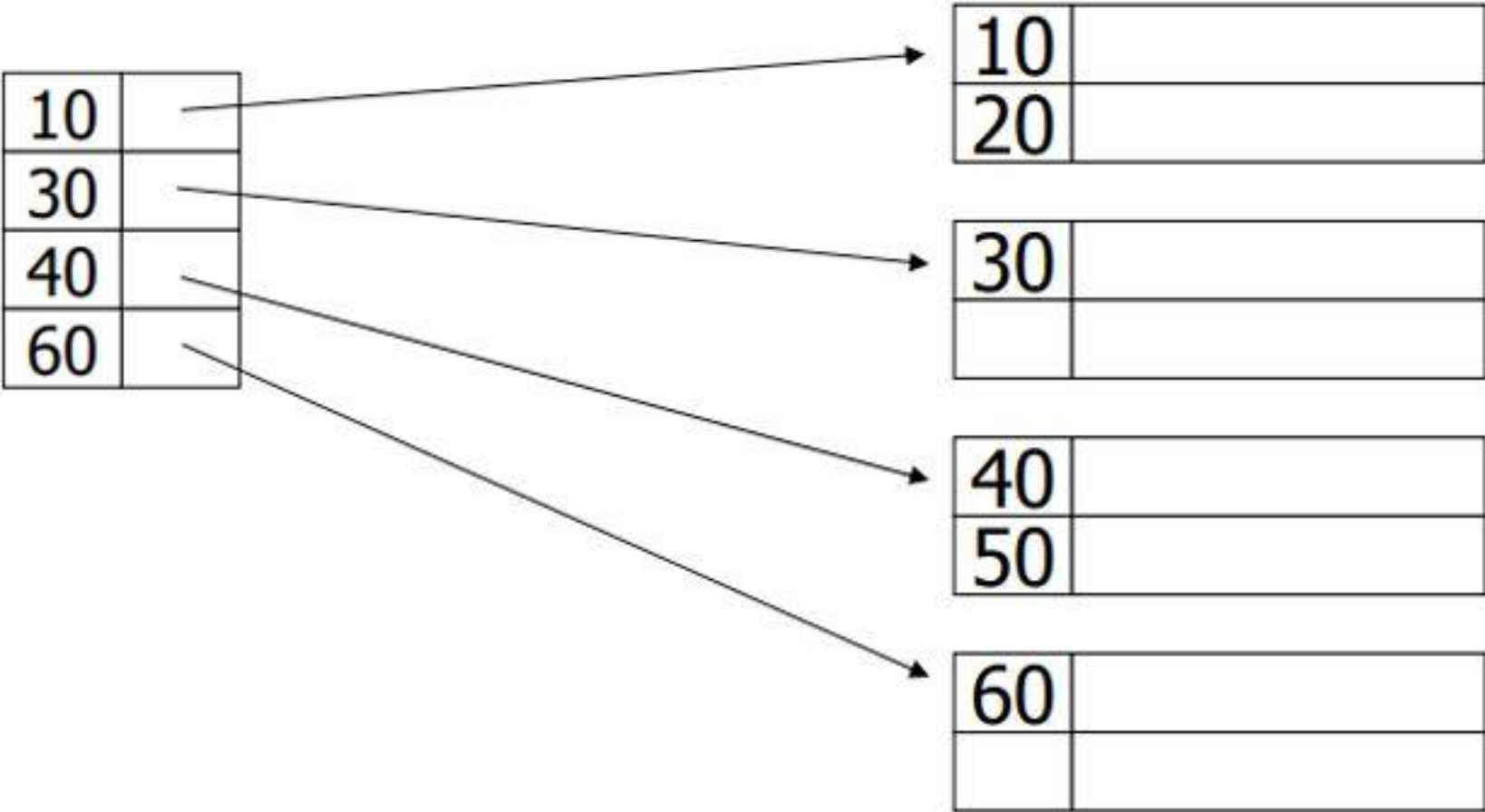
Insert 34



- nice!
we have free space
where we need it!

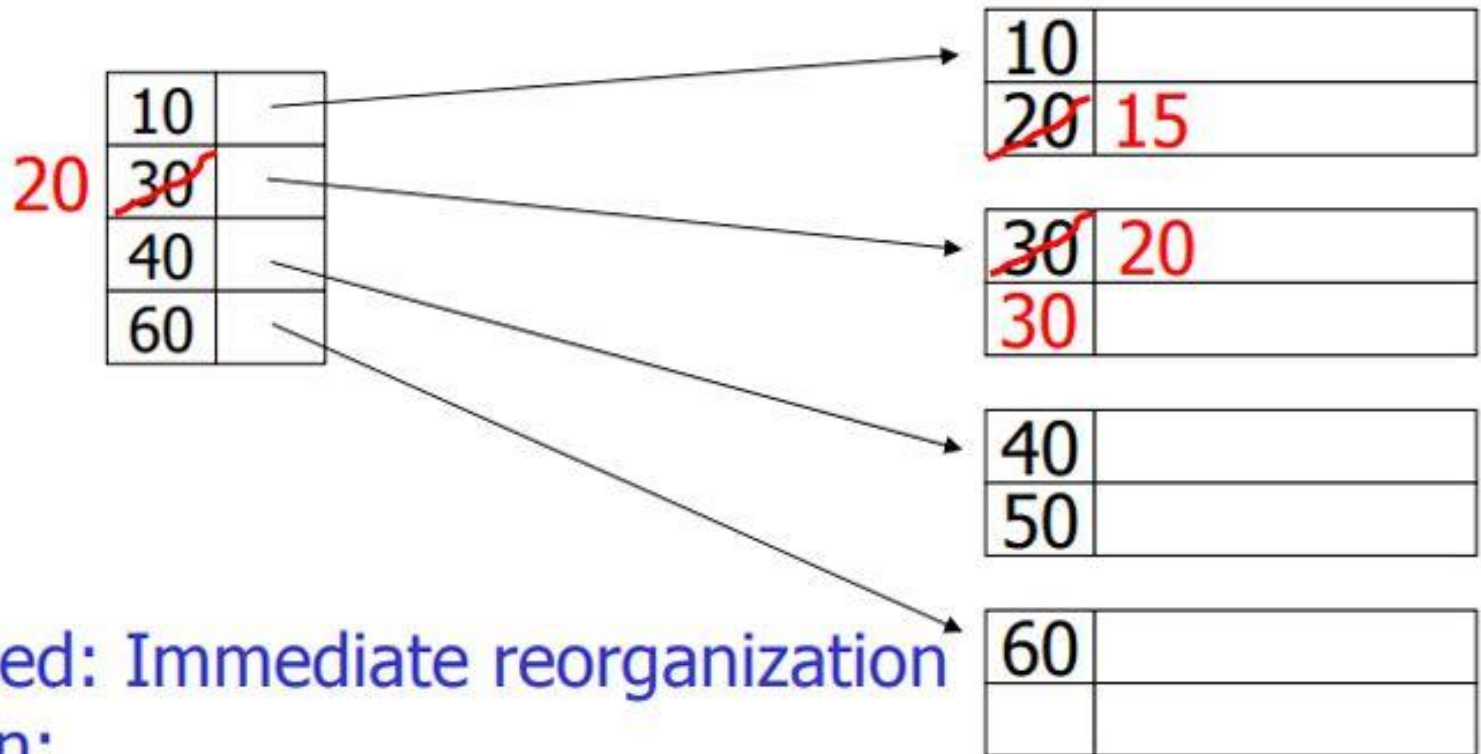
Insertion Into Sparse Index

Insert 15



Insertion Into Sparse Index

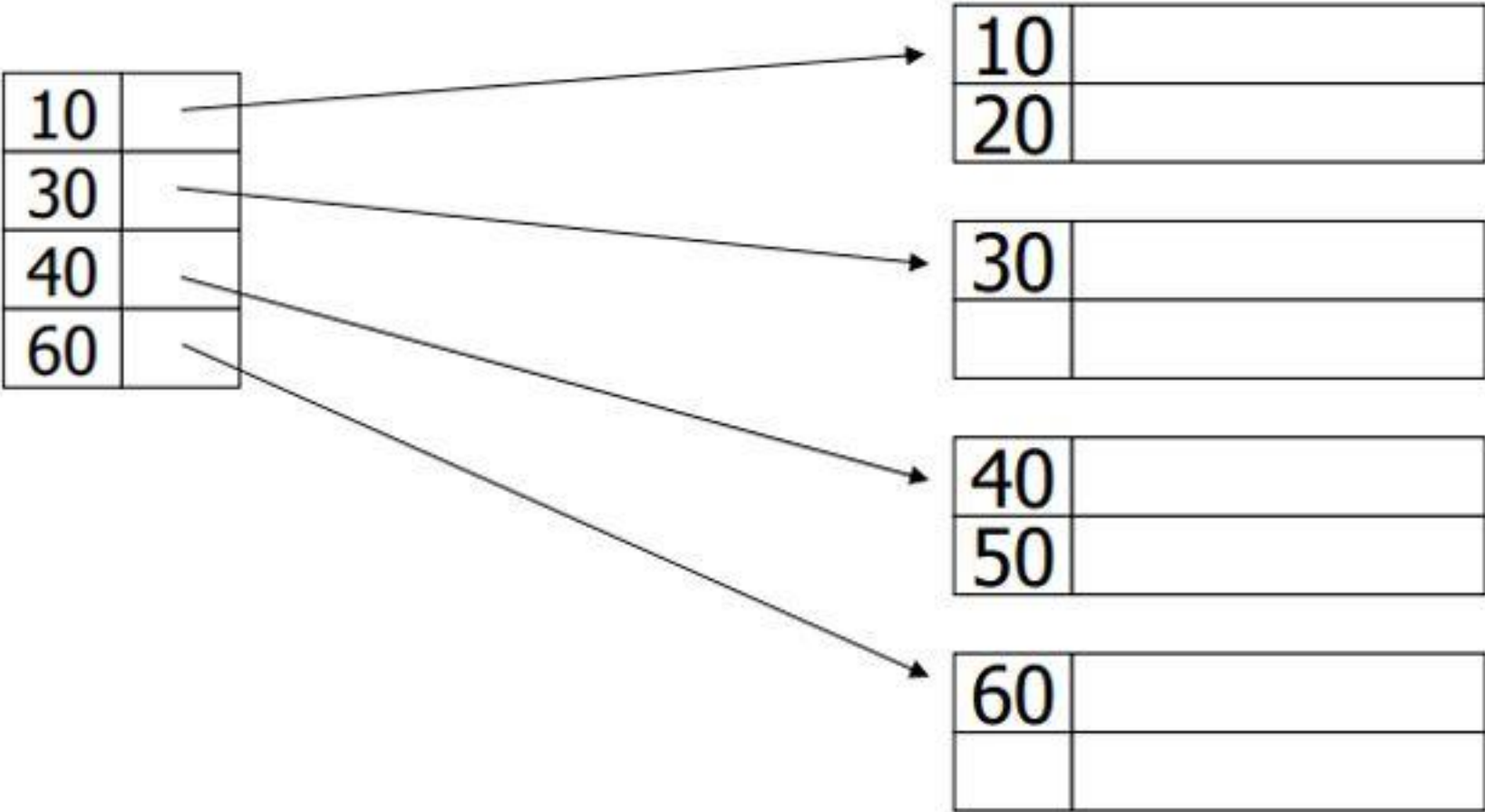
Insert 15



- Illustrated: Immediate reorganization
- Variation:
 - insert new block (chained)

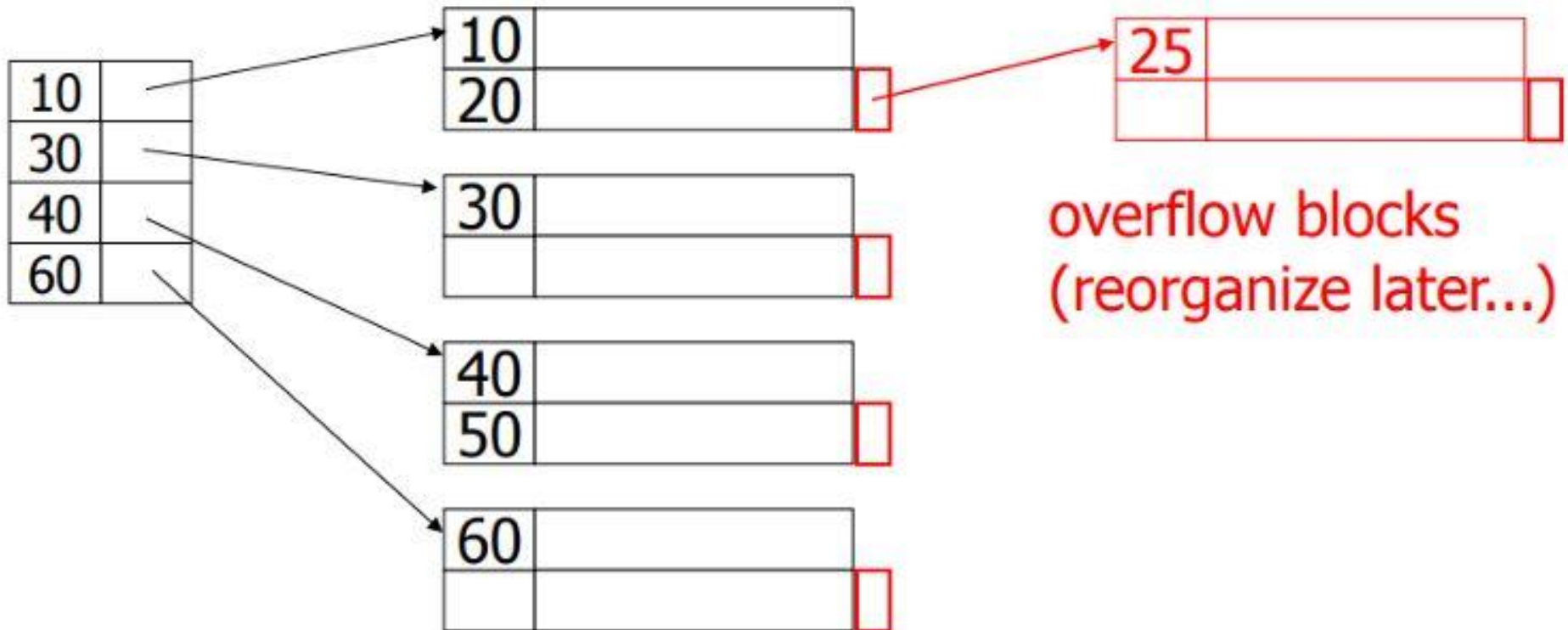
Insertion Into Sparse Index

Insert 25



Insertion Into Sparse Index

Insert 25



Insertion into Dense Index

- Similar.
- Often more expensive . . .

Conventional indices

- Advantage:
 - Simple.
 - Index is sequential file (good for scans).
- Disadvantage:
 - Inserts/deletes expensive